

Bewerberaufgabe: Lokales Kubernetes-Setup mit Let's Encrypt und Keycloak

Anforderungen im Detail

1. Lokales Kubernetes-Cluster

Ziel

Setze ein lokales Kubernetes-Cluster auf, das geeignet ist, um typische Plattform-Komponenten zu installieren und zu testen.

Das Cluster soll mindestens folgende Kubernetes-Ressourcen sinnvoll unterstützen:

- Namespaces
 - Deployments
 - StatefulSets
 - Services
 - Ingress
 - Secrets
 - ConfigMaps
 - PersistentVolumes / PersistentVolumeClaims
 - Custom Resource Definitions, insbesondere für cert-manager
-

1.1 Wahl der Kubernetes-Distribution

Du kannst frei entscheiden, welche lokale Kubernetes-Umgebung du verwendest.

Geeignete Optionen sind zum Beispiel:

- kind
- minikube
- k3d
- k3s lokal
- Docker Desktop Kubernetes
- Rancher Desktop
- MicroK8s
- eine andere geeignete lokale Kubernetes-Umgebung

Bitte dokumentiere, welche Lösung du gewählt hast und warum.

Erwartete Begründung

Beschreibe kurz:

- Warum du dich für diese Kubernetes-Variante entschieden hast
- Welche Vorteile sie für ein lokales Entwicklungssetup bietet
- Welche Einschränkungen sie hat
- Welche Versionen verwendet wurden

Beispiel:

```
1 Ich verwende kind, da es schnell lokal reproduzierbare Kubernetes-Cluste
2 Für diese Aufgabe ist kind ausreichend, da es Helm, Ingress Controller,
3 Eine Einschränkung besteht darin, dass LoadBalancer-Services nicht ohne
```

1.2 Cluster-Konfiguration

Das lokale Kubernetes-Cluster sollte nicht nur mit Standardwerten gestartet werden, sondern bewusst für diese Aufgabe konfiguriert sein.

Bitte dokumentiere:

- Wie das Cluster erstellt wird
- Welche Kubernetes-Version verwendet wird
- Wie viele Nodes erstellt werden
- Ob ein Single-Node- oder Multi-Node-Cluster verwendet wird
- Wie lokale Ports vom Host in das Cluster geleitet werden
- Wie Ingress-Zugriff ermöglicht wird
- Ob Storage unterstützt wird
- Wie das Cluster wieder gelöscht werden kann

Mindestanforderung

Ein Single-Node-Cluster ist ausreichend, sofern alle benötigten Komponenten darauf lauffähig sind.

Optional

Ein Multi-Node-Cluster ist willkommen, aber nicht erforderlich.

Bei einem Multi-Node-Cluster sollte kurz erklärt werden:

- Welche Rolle die einzelnen Nodes haben
- Ob Workloads gezielt platziert werden

- Ob NodePort, LoadBalancer oder Ingress verwendet wird
 - Wie lokale Ports erreichbar gemacht werden
-

1.3 Beispielhafte Cluster-Struktur

Die konkrete Umsetzung ist dir überlassen. Eine mögliche Struktur könnte so aussehen:

```
1 Lokaler Rechner
2   └─ Kubernetes-Cluster
3       └─ ingress-nginx Namespace
4           └─ Ingress Controller
5       └─ cert-manager Namespace
6           └─ cert-manager
7           └─ cert-manager-webhook
8           └─ cert-manager-cainjector
9       └─ keycloak Namespace
10          └─ Keycloak
11          └─ PostgreSQL
12          └─ Service
13          └─ Ingress
14          └─ TLS Certificate
15       └─ ggf. weitere Namespaces
```

1.4 Kubernetes-Kontext und Zugriff

Bitte stelle sicher, dass nach dem Cluster-Setup ein funktionierender `kubectl` -Kontext vorhanden ist.

Dokumentiere, wie geprüft werden kann, dass das Cluster erreichbar ist.

Beispiele:

```
1 kubectl cluster-info
2 kubectl get nodes
3 kubectl get namespaces
4 kubectl get pods -A
```

Die Ausgabe muss nicht vollständig in der Dokumentation enthalten sein. Es sollte aber beschrieben sein, welche Kommandos zur Validierung genutzt werden können.

1.5 Namespaces

Verwende sinnvolle Namespaces zur Trennung der Komponenten.

Mindestens empfohlen:

```
1 ingress-nginx
2 cert-manager
3 keycloak
```

Alternativ können andere Namespace-Namen verwendet werden, sofern sie dokumentiert sind.

Bitte beschreibe:

- Welche Namespaces erstellt werden
- Welche Komponenten in welchem Namespace laufen
- Warum diese Trennung gewählt wurde

Beispiel:

```
1 Der Namespace cert-manager enthält ausschließlich Komponenten für Zertif
2 Der Namespace keycloak enthält Keycloak, die Datenbank und die zugehörig
```

1.6 Netzwerk und Service-Modell

Beschreibe, wie Netzwerkzugriff innerhalb und außerhalb des Clusters funktioniert.

Bitte gehe insbesondere auf folgende Punkte ein:

- Welche Services verwendet werden
- Ob Services vom Typ `ClusterIP`, `NodePort` oder `LoadBalancer` verwendet werden
- Wie der Ingress Controller erreichbar gemacht wird
- Wie HTTP und HTTPS vom lokalen Rechner in das Cluster gelangen
- Welche Ports auf dem Host verwendet werden

Beispielhafte Ports:

```
1 HTTP: 80
2 HTTPS: 443
```

oder, falls Standardports nicht möglich sind:

```
1 HTTP: 8080
2 HTTPS: 8443
```

Falls Port-Mapping notwendig ist, dokumentiere dieses bitte ausführlich.

Beispiele:

```
1 kubectl port-forward -n ingress-nginx service/ingress-nginx-controller 8
2 kubectl port-forward -n ingress-nginx service/ingress-nginx-controller 8
```

oder bei kind:

```
1 nodes:
2   - role: control-plane
3     extraPortMappings:
4       - containerPort: 80
5         hostPort: 80
6         protocol: TCP
7       - containerPort: 443
8         hostPort: 443
```

1.7 Ingress Controller

Installiere oder aktiviere einen Ingress Controller.

Geeignete Optionen sind zum Beispiel:

- ingress-nginx
- Traefik
- HAProxy Ingress
- Istio Gateway
- ein anderer geeigneter Ingress Controller

Bitte dokumentiere:

- Welcher Ingress Controller verwendet wird
- Wie er installiert wird
- In welchem Namespace er läuft
- Wie er erreichbar gemacht wird
- Welche IngressClass verwendet wird
- Ob zusätzliche Annotationen erforderlich sind

Beispiel zur Validierung:

```
1 kubectl get ingressclass
2 kubectl get pods -n ingress-nginx
3 kubectl get svc -n ingress-nginx
```

1.8 DNS und lokale Hostnamen

Keycloak soll über einen Hostnamen erreichbar sein.

Beispiel:

```
1 keycloak.local.example
```

oder:

```
1 keycloak.127.0.0.1.nip.io
```

oder:

```
1 keycloak.<deine-domain>
```

Bitte dokumentiere, welche Variante du verwendest.

Falls `/etc/hosts` angepasst werden muss, gib die notwendigen Einträge an.

Beispiel:

```
1 | 127.0.0.1 keycloak.local.example
```

Falls du eine dynamische lokale DNS-Lösung wie `nip.io`, `sslip.io` oder eine eigene Domain verwendest, beschreibe kurz, wie diese funktioniert.

1.9 Storage

Keycloak benötigt je nach Konfiguration persistente Datenhaltung. Besonders relevant ist dies, wenn eine PostgreSQL-Datenbank verwendet wird.

Bitte dokumentiere:

- Ob persistenter Storage verwendet wird
- Welche StorageClass vorhanden ist
- Ob PersistentVolumeClaims erstellt werden
- Was bei einem Cluster-Neustart mit den Daten passiert
- Warum du dich für persistente oder nicht-persistente Datenhaltung entschieden hast

Validierung:

```
1 | kubectl get storageclass
2 | kubectl get pvc -A
3 | kubectl get pv
```

Für ein lokales Testsetup ist nicht-persistente Datenhaltung akzeptabel, sofern dies begründet wird.

Beispiel:

```
1 | Für diese Aufgabe wird nicht-persistenter Storage verwendet, da das Setu
2 | In einer realen Umgebung würde PostgreSQL mit persistentem Storage und B
```

1.10 Kubernetes Secrets und sensible Daten

Admin-Zugangsdaten, Datenbankpasswörter und andere sensible Werte sollten nicht im Klartext in der Dokumentation oder in Git abgelegt werden.

Bitte beschreibe:

- Welche Secrets verwendet werden

- Wie sie erstellt werden
- Wie Zugangsdaten ermittelt werden können
- Ob Secrets manuell, per Helm oder per Manifest erstellt werden
- Wie du vermeiden würdest, sensible Werte in einem echten Projekt in Git zu speichern

Beispiel:

```
1 kubectl get secret -n keycloak
2 kubectl get secret <secret-name> -n keycloak -o yaml
```

Falls Zugangsdaten aus einem Secret gelesen werden müssen, dokumentiere den Befehl.

Beispiel:

```
1 kubectl get secret keycloak-admin -n keycloak \
2 -o jsonpath="{.data.admin-password}" | base64 -d
```

1.11 Resource Requests und Limits

Setze für zentrale Komponenten sinnvolle Resource Requests und Limits oder begründe, warum du darauf verzichtest.

Bitte dokumentiere:

- CPU- und Memory-Anforderungen von Keycloak
- CPU- und Memory-Anforderungen der Datenbank
- Einschränkungen des lokalen Rechners
- Ob Limits bewusst gesetzt oder vermieden wurden

Beispiel:

```
1 resources:
2   requests:
3     cpu: 250m
4     memory: 512Mi
5   limits:
6     cpu: 1000m
7     memory: 1Gi
```

1.12 Health Checks

Bitte stelle sicher, dass Keycloak sinnvoll über Kubernetes überwacht werden kann.

Relevant sind insbesondere:

- Readiness Probe
- Liveness Probe

- Startup Probe

Dokumentiere:

- Ob das verwendete Helm Chart Probes bereitstellt
- Welche Endpunkte verwendet werden
- Wie geprüft werden kann, ob Keycloak bereit ist

Validierung:

```
1 kubectl get pods -n keycloak
2 kubectl describe pod <pod-name> -n keycloak
```

1.13 Cluster-Lifecycle

Bitte stelle Skripte oder Kommandos bereit, mit denen das Cluster reproduzierbar erstellt und gelöscht werden kann.

Empfohlen sind zum Beispiel:

```
1 scripts/create-cluster.sh
2 scripts/delete-cluster.sh
3 scripts/install-ingress.sh
4 scripts/install-cert-manager.sh
5 scripts/install-keycloak.sh
```

Alternativ kann ein `Makefile` verwendet werden.

Beispiel:

```
1 make cluster-create
2 make install
3 make validate
4 make cleanup
```

Bitte dokumentiere, welche Befehle in welcher Reihenfolge auszuführen sind.

2. Helm

Ziel

Alle zentralen Komponenten sollen per Helm installiert werden.

Erwartet wird:

- Verwendung offizieller oder etablierter Helm Charts
- Dokumentation der verwendeten Repositories
- Versionierung der verwendeten Charts

- Bereitstellung der verwendeten `values.yaml` -Dateien
 - Reproduzierbare Installation
-

2.1 Erwartete Helm-Komponenten

Mindestens folgende Komponenten sollten per Helm installiert werden:

- Ingress Controller, sofern nicht bereits durch die Kubernetes-Distribution bereitgestellt
- cert-manager
- Keycloak
- optional PostgreSQL

Bitte dokumentiere alle verwendeten Helm-Repositories.

Beispiel:

```
1 helm repo add jetstack https://charts.jetstack.io
2 helm repo add bitnami https://charts.bitnami.com/bitnami
3 helm repo update
```

2.2 Helm Values

Bitte lege alle relevanten Helm-Konfigurationen im Repository ab.

Beispielhafte Struktur:

```
1 helm/
2   ingress-nginx/
3     values.yaml
4   cert-manager/
5     values.yaml
6   keycloak/
7     values.yaml
```

Die `values.yaml` -Dateien sollten so aufgebaut sein, dass nachvollziehbar ist, welche Einstellungen bewusst vorgenommen wurden.

2.3 Helm-Installation dokumentieren

Bitte dokumentiere die verwendeten Helm-Befehle.

Beispiel:

```
1 helm upgrade --install cert-manager jetstack/cert-manager \
2   --namespace cert-manager \
3   --create-namespace \
4   --values helm/cert-manager/values.yaml
```

Für Keycloak entsprechend:

```
1 helm upgrade --install keycloak <chart-repository>/<chart-name> \  
2   --namespace keycloak \  
3   --create-namespace \  
4   --values helm/keycloak/values.yaml
```

3. cert-manager und Let's Encrypt

Ziel

Installiere cert-manager im Kubernetes-Cluster und konfiguriere ihn so, dass TLS-Zertifikate für Keycloak ausgestellt werden können.

3.1 cert-manager Installation

cert-manager soll per Helm installiert werden.

Erwartet wird:

- Installation inklusive CRDs
- eigener Namespace für cert-manager
- dokumentierte Helm-Version
- Validierung der Installation

Validierung:

```
1 kubectl get pods -n cert-manager  
2 kubectl get crds | grep cert-manager
```

3.2 Issuer oder ClusterIssuer

Konfiguriere einen `Issuer` oder `ClusterIssuer`.

Bitte dokumentiere:

- Ob du `Issuer` oder `ClusterIssuer` verwendest
- Warum du diese Variante gewählt hast
- Ob Let's Encrypt Staging oder Production genutzt wird
- Welche E-Mail-Adresse für Let's Encrypt verwendet wird
- Welche Challenge-Methode verwendet wird

Mögliche Challenge-Typen:

- HTTP-01
 - DNS-01
-

3.3 Let's Encrypt in lokaler Umgebung

Da es sich um ein lokales Kubernetes-Cluster handelt, ist Let's Encrypt nicht immer ohne Zusatzaufwand möglich.

Bitte gehe ausdrücklich auf diese Herausforderung ein.

Let's Encrypt muss die Domain öffentlich validieren können. Bei einem rein lokalen Hostnamen wie `keycloak.local` ist das nicht möglich.

Beschreibe daher eine der folgenden Varianten:

Variante A: Öffentliche Domain mit DNS-01 Challenge

Du verwendest eine echte Domain und validierst über DNS.

Bitte dokumentiere:

- Welche Domain verwendet wird
- Welcher DNS Provider verwendet wird
- Wie DNS-01 konfiguriert wurde
- Wie Secrets für DNS-Zugangsdaten gehandhabt werden

Variante B: Öffentliche Erreichbarkeit per Tunnel und HTTP-01 Challenge

Du verwendest zum Beispiel:

- ngrok
- cloudflared
- inlets
- einen eigenen Reverse Proxy
- eine andere Tunnel-Lösung

Bitte dokumentiere:

- Welche Tunnel-Lösung verwendet wird
- Wie der lokale Ingress öffentlich erreichbar gemacht wird
- Wie HTTP-01 funktioniert
- Welche Einschränkungen bestehen

Variante C: Lokale Alternative mit selbstsigniertem Zertifikat

Falls eine echte Let's-Encrypt-Ausstellung lokal nicht umgesetzt wird, dokumentiere bitte:

- Warum Let's Encrypt lokal nicht vollständig möglich ist
- Wie du dies erkannt hast
- Welche Alternative du lokal verwendest
- Wie das Setup in einer realen Umgebung mit Let's Encrypt funktionieren würde

Eine lokale Alternative kann zum Beispiel sein:

- self-signed Issuer in cert-manager
- lokale CA
- mkcert
- manuell erzeugtes TLS Secret

Diese Variante ist akzeptabel, wenn die Einschränkungen sauber erklärt werden.

3.4 Zertifikatsvalidierung

Bitte dokumentiere, wie geprüft werden kann, ob Zertifikate erfolgreich erstellt wurden.

Beispiele:

```
1 kubectl get clusterissuer
2 kubectl get issuer -A
3 kubectl get certificate -A
4 kubectl get certificaterequest -A
5 kubectl get order -A
6 kubectl get challenge -A
```

Weitere Details:

```
1 kubectl describe certificate <certificate-name> -n keycloak
2 kubectl describe challenge <challenge-name> -n keycloak
```

4. Keycloak Deployment

Ziel

Deploye Keycloak per Helm in das lokale Kubernetes-Cluster.

Keycloak soll anschließend über den konfigurierten Ingress per HTTPS erreichbar sein.

4.1 Helm Chart

Verwende ein geeignetes Helm Chart für Keycloak.

Bitte dokumentiere:

- Welches Helm Chart verwendet wird
 - Welche Chart-Version verwendet wird
 - Warum dieses Chart gewählt wurde
 - Welche Werte angepasst wurden
-

4.2 Keycloak-Konfiguration

Die Konfiguration sollte mindestens folgende Punkte abdecken:

- Admin-Benutzer
- Admin-Passwort oder Secret
- Hostname
- Proxy-/Ingress-Konfiguration
- HTTP-/HTTPS-Verhalten hinter dem Ingress
- Datenbankkonfiguration
- Resource Requests und Limits
- Probes
- Service-Konfiguration

Bitte stelle die verwendete `values.yaml` bereit.

4.3 Datenbank

Keycloak sollte idealerweise mit PostgreSQL betrieben werden.

Für diese Aufgabe sind zwei Varianten möglich:

Variante A: PostgreSQL innerhalb des Clusters

Dies ist die bevorzugte Variante für diese Aufgabe.

Bitte dokumentiere:

- Wie PostgreSQL installiert wird
- Ob PostgreSQL über dasselbe Helm Chart oder separat installiert wird
- Welche Secrets verwendet werden

- Ob persistenter Storage verwendet wird
- Wie Keycloak die Datenbank erreicht

Variante B: Entwicklungsmodus oder eingebettete Datenbank

Diese Variante ist nur für lokale Tests akzeptabel.

Bitte dokumentiere:

- Warum diese Variante gewählt wurde
 - Welche Einschränkungen sie hat
 - Warum sie nicht produktionsgeeignet ist
-

4.4 Zugriff auf Keycloak

Bitte dokumentiere:

- URL der Keycloak Admin Console
- Admin-Benutzer
- Wie das Admin-Passwort ermittelt wird
- Welche lokalen DNS-Einträge notwendig sind
- Wie HTTPS geprüft werden kann

Beispiel:

```
1 | URL: https://keycloak.local.example
```

Beispiel zur Prüfung:

```
1 | curl -vk https://keycloak.local.example
```

4.5 Keycloak-Funktionstest

Bitte beschreibe, wie geprüft werden kann, dass Keycloak funktioniert.

Mögliche Prüfschritte:

- Keycloak-Pod läuft erfolgreich
- Service ist erreichbar
- Ingress leitet korrekt weiter
- Admin Console ist erreichbar
- Login mit Admin-Zugangsdaten funktioniert
- TLS-Zertifikat wird ausgeliefert

- Keycloak zeigt keine offensichtlichen Fehler in den Logs

Beispiele:

```
1 kubectl get pods -n keycloak
2 kubectl get svc -n keycloak
3 kubectl get ingress -n keycloak
4 kubectl logs -n keycloak <keycloak-pod-name>
```

5. Ingress und HTTPS

Ziel

Keycloak soll nicht nur per Port-Forwarding erreichbar sein, sondern über einen Kubernetes Ingress.

5.1 Ingress-Konfiguration

Bitte stelle eine Ingress-Konfiguration bereit, entweder direkt als Kubernetes Manifest oder über Helm Values.

Der Ingress sollte enthalten:

- Hostname
- Service-Ziel
- TLS-Konfiguration
- IngressClass
- cert-manager Annotationen, falls verwendet

Beispielhafte Struktur:

```
1 ingress:
2   enabled: true
3   ingressClassName: nginx
4   hostname: keycloak.local.example
5   tls: true
```

5.2 HTTPS

Der Zugriff auf Keycloak soll über HTTPS erfolgen.

Bitte dokumentiere:

- Wo das TLS-Zertifikat erzeugt wird
- Welches Kubernetes Secret das Zertifikat enthält

- Wie der Ingress dieses Secret verwendet
- Wie der Browser oder `curl` das Zertifikat bewertet
- Ob es sich um ein echtes Let's-Encrypt-Zertifikat oder eine lokale Alternative handelt

Validierung:

```
1 kubectl get secret -n keycloak
2 kubectl describe ingress -n keycloak
3 curl -vk https://keycloak.local.example
```

6. Repository-Struktur

Bitte stelle deine Lösung in einem Git-Repository bereit.

Eine mögliche Struktur ist:

```
1 .
2 |__ README.md
3 |__ Makefile
4 |__ cluster/
5 |   |__ kind-config.yaml
6 |   |__ README.md
7 |__ helm/
8 |   |__ ingress-nginx/
9 |   |   |__ values.yaml
10 |   |__ cert-manager/
11 |   |   |__ values.yaml
12 |   |__ keycloak/
13 |   |   |__ values.yaml
14 |__ manifests/
15 |   |__ cert-manager/
16 |   |   |__ clusterissuer-letsencrypt-staging.yaml
17 |   |   |__ clusterissuer-letsencrypt-production.yaml
18 |   |__ keycloak/
19 |   |   |__ certificate.yaml
20 |   |   |__ ingress.yaml
21 |   |__ namespaces.yaml
22 |__ scripts/
23 |   |__ create-cluster.sh
24 |   |__ delete-cluster.sh
25 |   |__ install-ingress.sh
26 |   |__ install-cert-manager.sh
27 |   |__ install-keycloak.sh
28 |   |__ validate.sh
```

Diese Struktur ist nur ein Vorschlag. Eine andere Struktur ist akzeptabel, sofern sie verständlich dokumentiert ist.

7. Dokumentation

Die `README.md` im Repository sollte so geschrieben sein, dass eine andere technische Person das Setup ohne zusätzliche Erklärung nachvollziehen kann.

Sie sollte mindestens folgende Inhalte enthalten:

7.1 Voraussetzungen

Bitte liste alle benötigten Tools auf.

Beispiele:

- Docker
- kubectl
- Helm
- kind, minikube, k3d oder eine andere Kubernetes-Distribution
- openssl
- curl
- ggf. ngrok oder cloudflared
- ggf. Zugriff auf eine echte Domain
- ggf. Zugriff auf einen DNS Provider

Bitte gib nach Möglichkeit auch Versionen an.

Beispiele:

```
1 docker --version
2 kubectl version --client
3 helm version
4 kind version
```

7.2 Setup-Anleitung

Beschreibe Schritt für Schritt, wie das Setup erstellt wird.

Beispiel:

```
1 git clone <repository-url>
2 cd <repository-name>
3
4 ./scripts/create-cluster.sh
5 ./scripts/install-ingress.sh
6 ./scripts/install-cert-manager.sh
7 ./scripts/install-keycloak.sh
8 ./scripts/validate.sh
```

Alternativ:

```
1 make cluster-create
2 make install
3 make validate
```

7.3 Architekturüberblick

Beschreibe die Architektur deiner Lösung.

Bitte gehe auf folgende Punkte ein:

- Kubernetes-Distribution
- Cluster-Aufbau
- Namespaces
- Ingress Controller
- cert-manager
- Let's-Encrypt-Setup
- Keycloak
- Datenbank
- Storage
- Secrets
- Netzwerkzugriff
- DNS-Konfiguration

Ein Diagramm ist optional, aber willkommen.

7.4 Zugriff auf Keycloak

Dokumentiere:

- URL
- Hostname
- notwendige `/etc/hosts` -Einträge
- Admin-Benutzer
- Ermittlung des Admin-Passworts
- erwartetes Verhalten beim TLS-Zertifikat

Beispiel:

```
1 URL: https://keycloak.local.example
2 Admin User: admin
3 Admin Password: siehe Kubernetes Secret
```

7.5 Validierung

Bitte beschreibe, wie die Installation überprüft werden kann.

Beispiele:

```
1 kubectl get nodes
2 kubectl get pods -A
3 kubectl get ingress -A
4 kubectl get certificate -A
5 kubectl get clusterissuer
6 kubectl describe ingress keycloak -n keycloak
7 curl -vk https://keycloak.local.example
```

Die Validierung sollte zeigen:

- Cluster läuft
 - Ingress Controller läuft
 - cert-manager läuft
 - Keycloak läuft
 - Zertifikat wurde erstellt oder lokale Alternative ist aktiv
 - HTTPS-Zugriff funktioniert
-

7.6 Troubleshooting

Dokumentiere typische Fehler und mögliche Lösungen.

Beispiele:

Problem: Ingress ist nicht erreichbar

Mögliche Ursachen:

- Ingress Controller läuft nicht
- Hostname zeigt nicht auf `127.0.0.1`
- Port-Mapping fehlt
- falsche IngressClass
- Service-Name oder Service-Port ist falsch

Nützliche Befehle:

```
1 kubectl get ingress -A
2 kubectl describe ingress <name> -n <namespace>
3 kubectl get svc -A
4 kubectl logs -n ingress-nginx deployment/ingress-nginx-controller
```

Problem: Zertifikat wird nicht ausgestellt

Mögliche Ursachen:

- Domain ist nicht öffentlich erreichbar

- HTTP-01 Challenge kann nicht validiert werden
- DNS-01 Credentials fehlen oder sind falsch
- Let's Encrypt Rate Limits
- falscher Issuer oder ClusterIssuer
- cert-manager Annotation fehlt

Nützliche Befehle:

```
1 kubectl get certificate -A
2 kubectl describe certificate <name> -n <namespace>
3 kubectl get challenge -A
4 kubectl describe challenge <name> -n <namespace>
5 kubectl logs -n cert-manager deployment/cert-manager
```

Problem: Keycloak startet nicht

Mögliche Ursachen:

- Datenbank nicht erreichbar
- falsche Zugangsdaten
- zu wenig Memory
- fehlerhafte Hostname- oder Proxy-Konfiguration
- fehlende Secrets

Nützliche Befehle:

```
1 kubectl get pods -n keycloak
2 kubectl describe pod <pod-name> -n keycloak
3 kubectl logs <pod-name> -n keycloak
```

8. Annahmen und Einschränkungen

Bitte dokumentiere alle Annahmen und Einschränkungen.

Beispiele:

- verwendetes Betriebssystem
- verfügbare lokale Ressourcen
- verwendete Kubernetes-Version
- verwendete lokale Domain
- Umgang mit DNS
- Umgang mit TLS
- Einschränkungen von Let's Encrypt in lokalen Umgebungen

- Unterschiede zwischen lokalem Setup und produktiver Umgebung

Beispiel:

```
1 Das Setup wurde unter macOS mit Docker Desktop getestet.  
2 Der Ingress Controller wird über lokale Port-Mappings auf Port 80 und 443  
3 Für lokale Tests wird ein selbstsigniertes Zertifikat verwendet.  
4 In einer produktiven Umgebung würde Let's Encrypt per DNS-01 Challenge m
```

9. Bewertungskriterien

9.1 Technische Umsetzung

Bewertet wird insbesondere:

- Funktionierendes lokales Kubernetes-Cluster
- Nachvollziehbare Cluster-Konfiguration
- Saubere Namespace-Struktur
- Funktionierender Ingress Controller
- Installation der Komponenten per Helm
- Funktionierende cert-manager-Installation
- Nachvollziehbare Let's-Encrypt- oder TLS-Konfiguration
- Funktionierendes Keycloak Deployment
- Zugriff auf Keycloak über HTTPS
- Sinnvolle Verwendung von Secrets
- Sinnvolle Resource-Konfiguration

9.2 Kubernetes-Verständnis

Bewertet wird insbesondere:

- Verständnis von Deployments, StatefulSets, Services und Ingress
 - Umgang mit Namespaces
 - Umgang mit Kubernetes Secrets
 - Umgang mit PersistentVolumes und PersistentVolumeClaims
 - Verständnis von IngressClass und Ingress Controller
 - Verständnis von TLS Secrets
 - Verständnis von cert-manager Ressourcen
 - Fähigkeit zur Fehleranalyse mit `kubectl describe`, `kubectl logs` und Events
-

9.3 Plattform-Engineering-Denken

Bewertet wird insbesondere:

- Reproduzierbarkeit
- Automatisierung
- Wartbarkeit
- klare Strukturierung des Repositorys
- saubere Trennung von Konfiguration und Ausführung
- nachvollziehbare technische Entscheidungen
- realistische Einschätzung lokaler Einschränkungen
- sicherer Umgang mit Zugangsdaten
- produktionsnahe Denkweise, auch wenn das Setup lokal ist

9.4 Dokumentation

Bewertet wird insbesondere:

- Verständliche README
- klare Setup-Schritte
- nachvollziehbare Architekturbeschreibung
- dokumentierte Validierung
- dokumentiertes Troubleshooting
- klare Beschreibung von Annahmen und Einschränkungen

10. Optionale Zusatzaufgaben

Die folgenden Punkte sind optional, können aber positiv in die Bewertung einfließen:

- Makefile für Setup, Validierung und Cleanup
- automatisiertes Cleanup des gesamten Setups
- GitOps-Ansatz mit Argo CD oder Flux
- Verwendung von Terraform, OpenTofu oder Ansible
- eigene lokale CA mit mkcert
- PostgreSQL als separates Helm Release
- Backup- und Restore-Konzept für Keycloak
- Monitoring-Grundlagen mit Prometheus und Grafana

- Logging-Konzept
 - NetworkPolicies
 - PodSecurity-Konfiguration
 - ResourceQuotas und LimitRanges
 - Dokumentation eines produktionsnahen Zielbilds
 - Trennung zwischen lokaler, staging- und produktionsnaher Konfiguration
-

11. Erwartetes Ergebnis

Am Ende sollte ein Repository vorliegen, mit dem das Setup reproduzierbar lokal gestartet werden kann.

Eine andere technische Person sollte anhand der Dokumentation in der Lage sein:

1. die Voraussetzungen zu installieren,
 2. das Kubernetes-Cluster zu erstellen,
 3. die benötigten Komponenten zu deployen,
 4. Keycloak über HTTPS aufzurufen,
 5. die Installation zu validieren,
 6. typische Fehler zu analysieren,
 7. das Setup wieder zu entfernen.
-

12. Hinweise

Es ist ausdrücklich erlaubt, technische Einschränkungen eines lokalen Kubernetes-Setups zu dokumentieren und eine nachvollziehbare Alternative zu verwenden.

Wichtig ist, dass die getroffenen Entscheidungen erklärt werden.

Beispiel:

```
1 Let's Encrypt kann keine Zertifikate für rein lokale Domains wie keycloa
2 Daher verwende ich lokal eine selbstsignierte CA über cert-manager.
3 Für eine produktionsnahe Umgebung beschreibe ich zusätzlich, wie die Aus
```

13. Fragen

Bei Fragen zur Aufgabe kannst du dich jederzeit melden:

m.neuheiser@microsynetics.de